Zadanie 02 - Room Booking API: architektura i reguły biznesowe

Założenia

Rozwijamy Room Booking API z zadania 01. Punktem wyjścia jest Twoje rozwiązanie zestawu 01.

Na tym etapie **nadal nie używamy bazy danych**. Dane przechowujemy w pamięci, ale **wyłącznie w warstwie repository**.

Zmienia się model rezerwacji. Zamiast jednego pola `date` rezerwacja ma teraz **przedział czasu**:

```
{
  "id": 1,
  "roomId": 1,
  "reservedBy": "Jan Kowalski",
  "participants": 20,
  "startTime": "2026-11-10T10:00:00.000Z",
  "endTime": "2026-11-10T12:00:00.000Z",
  "status": "CONFIRMED",
  "createdAt": "2026-10-20T08:15:00.000Z",
  "cancelledAt": null
}
```

Wszystkie daty w API są w formacie **ISO 8601**. Na wejściu akceptujemy datę ze strefą czasową (`Z` albo np. `+01:00`), na wyjściu zawsze zwracamy UTC (`...Z`).

Wymagane technologie: Node.js, ExpressJS 5, Zod.

Za cały zestaw można uzyskać maksymalnie **5 punktów**.

Każde zadanie jest warte **1 punkt**.

Zadanie 1 - Architektura warstwowa - 1 pkt

Przebuduj aplikację na architekturę warstwową opisaną w materiale 02.

Wymagana struktura (nazwy plików mogą się nieznacznie różnić):

```
src/
├── app.js
├── server.js
├── errors/
├── middleware/
└── modules/
    ├── rooms/
    │   ├── rooms.routes.js
    │   ├── rooms.controller.js
    │   ├── rooms.service.js
    │   ├── rooms.repository.memory.js
    │   ├── rooms.schemas.js
    │   └── rooms.mapper.js
    └── reservations/
        └── ... (analogicznie)
```

Wymagania

- `app.js` eksportuje funkcję `createApp(...)`, która **buduje** aplikację, ale jej **nie uruchamia**,
- `server.js` jest jedynym miejscem, które woła `app.listen()` i tworzy konkretne repositories,
- repositories są przekazywane do services jako parametry (dependency injection), a nie importowane w services,
- services **nie importują** niczego z Expressa i **nie dostają** `req` ani `res`,
- tablice z danymi istnieją **wyłącznie** w plikach `*.repository.memory.js`,
- wszystkie metody repositories zwracają `Promise` (`async`),
- ID nowych rezerwacji generowane są licznikiem w repository, **nie** przez `array.length + 1`,
- wszystkie endpointy z zadania 01 działają jak wcześniej (z uwzględnieniem nowego modelu z `startTime` / `endTime`).

W sprawozdaniu pokaż drzewo katalogów oraz kod jednego pełnego przepływu: route → controller → service → repository.

Zadanie 2 - Validation z Zod - 1 pkt

Zastąp ręczne `if`-y walidujące schematami Zod oraz uniwersalnym middleware `validate`.

Wymagania

- jeden middleware `validate({ params, query, body })` używany we **wszystkich** endpointach przyjmujących dane,
- schemat dla `POST /api/reservations`:
  - `roomId` - dodatnia liczba całkowita,
  - `reservedBy` - string, po `trim()` długość 3-100,
  - `participants` - dodatnia liczba całkowita,
  - `startTime`, `endTime` - data ISO 8601 ze strefą czasową,
  - **nieznane pola są odrzucane** (np. klient nie może przesłać `status` ani `id`),
- `:id` w URL walidowane jako dodatnia liczba całkowita (`GET /api/rooms/abc` → `400`, a nie `404`),
- `roomId` w query (`GET /api/reservations?roomId=...`) walidowane tak samo,
- controllery korzystają z danych **po** walidacji (`req.validated`), a nie z surowego `req.body`,
- odpowiedź `400` zawiera **wszystkie** błędy naraz w formacie:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Validation failed",
    "details": [
      { "location": "body", "field": "participants", "message": "..." },
      { "location": "body", "field": "endTime", "message": "..." }
    ],
    "requestId": "..."
  }
}
```

W sprawozdaniu pokaż request z **co najmniej trzema** błędami naraz (w tym nieznanym polem) i

odpowiedź API.

Zadanie 3 - Global error handling - 1 pkt

Zaimplementuj centralną obsługę błędów.

Wymagania

- hierarchia klas błędów (`AppError` + minimum `ValidationError`, `NotFoundError`, `ConflictError`),
- **żaden** controller ani service nie wywołuje `res.status(4xx)` bezpośrednio - błędy są rzucane i obsługiwane w jednym error middleware,
- controllery **nie zawierają** `try/catch` (wykorzystaj mechanizm Express 5),
- każda odpowiedź z błędem ma ten sam format: `{ error: { code, message, details?, requestId } }`,
- nieistniejąca ścieżka (np. `GET /api/nope`) → `404` w formacie JSON (a nie domyślny HTML Expressa),
- niepoprawny JSON w body (np. `{"roomId": 1,`) → `400` z kodem `INVALID_JSON`,
- nieoczekiwany błąd → `500` z ogólnym komunikatem; **stack trace nie trafia do klienta**, tylko do logów,
- middleware logujący, który:
  - nadaje każdemu requestowi `requestId` (lub przejmuje go z nagłówka `X-Request-Id`),
  - zwraca `requestId` w nagłówku `X-Request-Id` odpowiedzi,
  - po zakończeniu requestu loguje: `requestId`, method, URL, status code, czas wykonania w ms.

Aby pokazać obsługę `500`, dodaj **tymczasowy** endpoint, np. `GET /api/debug/crash`, który rzuca zwykły `new Error("boom")`. W sprawozdaniu pokaż odpowiedź klienta oraz log servera dla tego samego `requestId`. Endpoint możesz potem usunąć.

Zadanie 4 - Business rule: brak nakładających się rezerwacji

- 1 pkt

Sala nie może mieć dwóch **aktywnych** (`CONFIRMED`) rezerwacji, których przedziały czasu się nakładają.

Jak sprawdzić, czy dwa przedziały się nakładają?

Traktujemy przedziały jako **półotwarte**: `[startTime, endTime)` - początek należy do przedziału, koniec nie.

Dwa przedziały `A` i `B` nakładają się wtedy i tylko wtedy, gdy:

```
A.start < B.end  AND  B.start < A.end
```

Przykłady dla istniejącej rezerwacji `10:00-12:00`:

Nowa rezerwacja	Wynik	Dlaczego
`12:00-13:00`	OK	styka się, ale nie nakłada (12:00 nie należy do `[10:00, 12:00)`)
`08:00-10:00`	OK	j.w.
`11:00-13:00`	409	nakłada się na 11:00-12:00
`09:00-13:00`	409	zawiera całą istniejącą rezerwację
`10:30-11:00`	409	zawiera się w istniejącej rezerwacji
`10:00-12:00` w **innej** sali	OK	inna sala
`10:00-12:00`, gdy istniejąca jest `CANCELLED`	OK	anulowane rezerwacje nie blokują sali

Wymagania

- nakładająca się rezerwacja → `409 Conflict` z kodem `ROOM_ALREADY_BOOKED`,
- `endTime` musi być późniejsze niż `startTime` → `400`,
- rezerwacja trwa minimum **15 minut**, maksimum **8 godzin** → `400`,
- `startTime` nie może być w przeszłości → `400`,
- reguła pojemności sali z zadania 01 nadal działa (`409`),
- wszystkie powyższe reguły znajdują się w **service**, nie w controllerze ani w schemacie Zod,
- aktualny czas jest **wstrzykiwany** do service (`now = () => new Date()`), a nie wywoływany bezpośrednio przez `new Date()` w logice.

> **Podpowiedź:** porównania dat rób na timestampach (`new Date(x).getTime()`), a nie na stringach. `"2026-11-10T10:00:00+01:00"` i `"2026-11-10T09:00:00Z"` to **ten sam** moment, choć stringi są różne.

W sprawozdaniu pokaż **wszystkie** przypadki z tabeli powyżej (screenshoty albo jeden plik `.http` z wynikami).

Zadanie 5 - Anulowanie jako state machine + response DTO - 1 pkt

Część A - reguły anulowania

Rozbuduj `PATCH /api/reservations/:id/cancel`.

Dozwolone przejścia statusów:

```
CONFIRMED ---> CANCELLED
```

`CANCELLED` jest stanem końcowym.

Wymagania:

- przejścia statusów zapisz jako **dane** (obiekt / mapa dozwolonych przejść), a nie jako ciąg `if`

-ów,

- rezerwacji, która **już się rozpoczęła** (`startTime <= now`), nie można anulować → `409` z kodem `RESERVATION_ALREADY_STARTED`,
- ponowne anulowanie rezerwacji `CANCELLED` zwraca `200` i tę samą rezerwację bez zmian (operacja **idempotentna**) - `cancelledAt` nie może się zmienić przy drugim wywołaniu,
- przy anulowaniu ustawiane jest pole `cancelledAt`.

Część B - response DTO

- każdy zasób ma mapper (`toRoomResponse`, `toReservationResponse`), który zwraca **jawnie wymienione** pola,
- model wewnętrzny rezerwacji zawiera dodatkowe pole `internalNote` (np. `"created via API v2"`), które **nie może** pojawić się w żadnej odpowiedzi API,
- daty zawsze w formacie ISO 8601 UTC, brakujące wartości jako `null`,
- `POST /api/reservations` zwraca `201` oraz nagłówek `Location: /api/reservations/{id}`.

W sprawozdaniu pokaż: anulowanie przyszłej rezerwacji, próbę anulowania rozpoczętej rezerwacji, podwójne anulowanie (z tym samym `cancelledAt`) oraz nagłówki odpowiedzi `201`.

> **Jak przetestować "rezerwację, która już się rozpoczęła", skoro nie można utworzyć rezerwacji w przeszłości?** Dodaj ją do danych startowych w repository (seed). To jest dokładnie ta sytuacja, którą reguła ma obsłużyć - rezerwacja była poprawna w momencie tworzenia, ale czas minął.

Punktacja

Zadanie	Punkty
Architektura warstwowa	1 pkt
Validation z Zod	1 pkt
Global error handling + logging	1 pkt

Brak nakładających się rezerwacji	1 pkt
Anulowanie (state machine) + DTO	1 pkt
Razem	**5 pkt**

Co oddać?

Przygotuj sprawozdanie w formacie PDF.

Powinno ono zawierać:

- drzewo katalogów projektu,
- fragmenty najważniejszego kodu (service z regułami, middleware `validate`, error handler),
- screenshoty potwierdzające działanie endpointów,
- przykłady poprawnych requestów,
- przykłady obsłużonych błędów: `400`, `404`, `409`, `500`,
- krótki opis rozwiązania.

Nie ma potrzeby umieszczania screenshotu całego kodu.

Najważniejsze jest pokazanie, że API działa zgodnie z wymaganiami.